

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Task No.: 5

Student Name: EMANDI HARI CHARAN

Roll No.: A24126552080

Date:

1. TITLE

Project-Based Learning — Interactive Product Calculator, Node.js File Management, Express.js Student Server, and DOM Manipulation

2. AIM

To implement four project-based learning technical solutions: (1) an interactive product pricing calculator using JavaScript event listeners and real-time DOM updates without page refreshes, (2) server-side file management utilizing Node.js built-in File System (fs) and readline modules, (3) a REST-style student API server configured using Express.js with custom routing, and (4) comprehensive DOM manipulation demonstrating dynamic changes to content, styling, and HTML attributes.

3. OBJECTIVE

The objectives of this experiment are:

- To combine JavaScript functions, numeric parsing, and real-time input event listeners for dynamic UI updates.
- To understand server-side asynchronous execution and command-line I/O using the Node.js readline interface.
- To execute file creation, writing, appending, and reading using the Node.js File System (fs) module.
- To configure an Express.js HTTP application server and implement structured routing (/ , /students, /about).
- To deliver JSON payloads from an Express API route and handle client HTTP GET requests.
- To manipulate DOM element textContent, inline style properties, and custom HTML attributes (setAttribute, getAttribute).

4. THEORY

This project-based learning task spans client-side interactivity, server runtime fundamentals, and web application architecture:

- **Interactive Client Updates (Question 1):** Demonstrates event-driven UI updates. The input event triggers whenever user enters or changes values, instantly recalculating the total without requiring button submissions or HTTP round-trips.

- Node.js File System Runtime (Question 2): Node.js executes JavaScript on the server outside browser sandboxes. The built-in fs module allows scripts to interact with the host operating system's file system, performing synchronous or asynchronous file writes, appends, and reads.
- Express.js Routing Architecture (Question 3): Express is a minimal and flexible Node.js web application framework. It provides routing mechanics to map HTTP methods (GET) and URL paths to callback handlers that construct and send HTML or serialized JSON responses.
- Document Object Model Manipulation (Question 4): The DOM interface allows script-based modification of living webpage elements. Using textContent, style, and setAttribute, applications update content, CSS styles, and metadata dynamically in memory.

5. ALGORITHM / PROCEDURE

Question 1: Interactive Product Calculator

1. Design HTML layout with product name, price, and quantity inputs, along with a total display box.
2. In JavaScript, query input elements and attach input event listeners to recalculate dynamically.
3. In calculateTotal(): parse price (parseFloat) and quantity (parseInt). Compute total = price * quantity. Update DOM textContent using toFixed(2).

Question 2: Node.js File Management

4. Initialize readline interface connected to process.stdin and process.stdout.
5. Prompt user for filename; prompt user for initial content. Write file using fs.writeFileSync().
6. Prompt user for additional content. Append to file using fs.appendFileSync().
7. Read final contents using fs.readFileSync(filename, 'utf8') and log output to terminal.

Question 3: Express.js Student Server

8. Initialize Express application and define listening port 3000.
9. Create student data collection containing 5 student objects with id, name, and course.
10. Define routes: GET '/' returning welcome HTML, GET '/students' returning JSON student list, GET '/about' returning application details.
11. Start server with app.listen() and verify endpoints via web browser.

Question 4: Document Object Model Demonstration

12. Build an explanation area and a target box element with control buttons.
13. Attach event listeners: changeContentBtn updates textContent; changeStyleBtn modifies background, color, and borderRadius; changeAttrBtn updates title attribute using setAttribute().
14. Implement resetBtn to revert element text, inline styles, and attributes to original defaults.

6. PROGRAM

Question 1: Interactive Product Calculator

Question1/index.html

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Product Calculator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="calculator-container">
    <h1>Product Price Calculator</h1>

    <div class="input-group">
      <label for="productName">Product Name:</label>
      <input type="text" id="productName" placeholder="e.g., Laptop">
    </div>

    <div class="input-group">
      <label for="productPrice">Price ($):</label>
      <input type="number" id="productPrice" min="0" placeholder="0">
    </div>

    <div class="input-group">
      <label for="productQuantity">Quantity:</label>
      <input type="number" id="productQuantity" min="1" value="1">
    </div>

    <div class="result-box">
      <h2>Total Cost</h2>
      <p id="totalDisplay">Total for <span id="displayName">Product</span>: $<span
id="displayTotal">0.00</span></p>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>

```

Question1/style.css

```

body {
  font-family: Arial, sans-serif;
  background-color: #f4f7f6;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}

.calculator-container {
  background-color: #fff;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}

```

```
    width: 350px;
}

h1 {
    font-size: 1.5em;
    color: #333;
    text-align: center;
    margin-bottom: 20px;
}

.input-group {
    margin-bottom: 15px;
}

label {
    display: block;
    margin-bottom: 5px;
    color: #555;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 5px;
    box-sizing: border-box;
    font-size: 16px;
}

input:focus {
    border-color: #007bff;
    outline: none;
}

.result-box {
    margin-top: 25px;
    background-color: #e9ecef;
    padding: 15px;
    border-radius: 5px;
    text-align: center;
}

h2 {
    margin-top: 0;
    color: #333;
    font-size: 1.2em;
}

p {
    font-size: 1.1em;
    color: #007bff;
    font-weight: bold;
    margin: 0;
}
```

Question1/script.js

```
// Get all the HTML elements we need to interact with
const nameInput = document.getElementById('productName');
const priceInput = document.getElementById('productPrice');
const quantityInput = document.getElementById('productQuantity');
const displayName = document.getElementById('displayName');
const displayTotal = document.getElementById('displayTotal');

// Function to calculate and update the total price
function calculateTotal() {
  // Get the values from the inputs
  // We use parseFloat for price to handle decimals, and parseInt for quantity
  const price = parseFloat(priceInput.value) || 0;
  const quantity = parseInt(quantityInput.value) || 0;
  let name = nameInput.value;

  // Default to 'Product' if the user hasn't typed a name
  if (name.trim() === '') {
    name = 'Product';
  }

  // Calculate total
  const total = price * quantity;

  // Update the HTML display
  displayName.textContent = name;

  // .toFixed(2) ensures the total has 2 decimal places (e.g., 10.50)
  displayTotal.textContent = total.toFixed(2);
}

// Add event listeners to trigger the calculation whenever the input changes
// The 'input' event fires every time the user types or changes the value
nameInput.addEventListener('input', calculateTotal);
priceInput.addEventListener('input', calculateTotal);
quantityInput.addEventListener('input', calculateTotal);

// Run calculation initially to set the default state on page load
calculateTotal();
```

Question 2: Node.js File Management

Question2/app.js

```
// Import required built-in Node.js modules
const fs = require('fs');
const readline = require('readline');

// Set up readline interface to read input from the terminal
const rl = readline.createInterface({
  input: process.stdin,
```

```

    output: process.stdout
  });

  console.log("=====");
  console.log("    Node.js File Management");
  console.log("=====\\n");

  // Step 1: Ask the user for a filename
  rl.question('Enter a filename (e.g., myData.txt): ', (filename) => {

    // Basic validation to ensure a filename was entered
    if (!filename.trim()) {
      console.log("[ERROR] Filename cannot be empty!");
      rl.close();
      return; // Exit the program
    }

    // Step 2: Ask the user for content to write into the file
    rl.question('Enter content to write to the file: ', (initialContent) => {

      try {
        // Write the content to the file
        // fs.writeFileSync creates the file if it doesn't exist, and overwrites if it
does
        fs.writeFileSync(filename, initialContent);
        console.log(`\\n[SUCCESS] File '${filename}' created/written successfully.`);
      } catch (error) {
        console.error(`[ERROR] Could not write to file: ${error.message}`);
        rl.close();
        return;
      }

      // Step 3: Ask the user for additional content to append
      rl.question('Enter additional content to append to the file: ', (appendContent) =>
{

        try {
          // Append the content. We add a newline character (\\n) so it starts on a
new line
          fs.appendFileSync(filename, '\\n' + appendContent);
          console.log(`[SUCCESS] Additional content appended to '${filename}'.`);
        } catch (error) {
          console.error(`[ERROR] Could not append to file: ${error.message}`);
          rl.close();
          return;
        }

        // Step 4: Read the final contents of the file and display it
        console.log(`\\n--- Final Contents of ${filename} ---`);
        try {
          const finalContents = fs.readFileSync(filename, 'utf8');
          console.log(finalContents);
        } catch (error) {
          console.error(`[ERROR] Could not read file: ${error.message}`);
        }
      }
    }
  }

```

```

        console.log("-----");

        // Close the readline interface and end program
        rl.close();
    });
});
});

```

Question 3: Express.js Student Server

Question3/package.json

```

{
  "name": "express-student-server",
  "version": "1.0.0",
  "description": "Basic Express.js server for PBL",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.22.3"
  }
}

```

Question3/app.js

```

// Import the express module
const express = require('express');

// Create an instance of an Express application
const app = express();

// Define a port number for the server to listen on
const PORT = 3000;

// Dummy data: List of at least five students
const students = [
  { id: 1, name: "Alice Johnson", course: "Computer Science" },
  { id: 2, name: "Bob Smith", course: "Information Technology" },
  { id: 3, name: "Charlie Brown", course: "Mechanical Engineering" },
  { id: 4, name: "Diana Prince", course: "Electrical Engineering" },
  { id: 5, name: "Evan Wright", course: "Civil Engineering" }
];

// Route 1: Home route ( '/')
// HTTP Method: GET
app.get('/', (req, res) => {
  res.send(`
    <div style="font-family: Arial, sans-serif; text-align: center; margin-top: 50px;">
      <h1>Welcome to the Student API Server!</h1>
      <p>Available routes:</p>
  `);
});

```

```

        <ul style="list-style-type: none; padding: 0;">
          <li><a href="/students" style="color: blue; text-decoration:
none;">/students</a> - View Student List</li>
          <li><a href="/about" style="color: blue; text-decoration: none;">/about</a>
- About Application</li>
        </ul>
      </div>
    `);
  });

// Route 2: Students route ('/students')
// HTTP Method: GET
// Returns the list of students in JSON format
app.get('/students', (req, res) => {
  // res.json() automatically formats the array/object as a JSON response
  // and sets the correct headers for JSON
  res.json(students);
});

// Route 3: About route ('/about')
// HTTP Method: GET
app.get('/about', (req, res) => {
  res.send(`
    <div style="font-family: Arial, sans-serif; padding: 20px;">
      <h1>About This Application</h1>
      <p>This is a basic Express.js server built for a Project-Based Learning (PBL)
task.</p>
      <p><strong>What it demonstrates:</strong></p>
      <ul>
        <li>Server setup and listening on a port</li>
        <li>Routing (/, /students, /about)</li>
        <li>Handling HTTP GET requests</li>
        <li>Responding with HTML and JSON data</li>
      </ul>
      <br>
      <a href="/">Back to Home</a>
    </div>
  `);
});

// Start the server and listen on the specified port
app.listen(PORT, () => {
  console.log(`\n=====`);
  console.log(`  Server is successfully running!`);
  console.log(`  Open your browser and visit:`);
  console.log(`    http://localhost:${PORT}`);
  console.log(`=====\n`);
});

```

Question 4: Document Object Model Demonstration

[Question4/index.html](#)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Manipulation Demo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Understanding the DOM</h1>

    <!-- Explanation of DOM -->
    <div class="explanation">
      <h2>What is the DOM?</h2>
      <p>The <strong>Document Object Model (DOM)</strong> is a programming interface
for web documents. It represents the page so that programs can change the document
structure, style, and content. The DOM represents the document as nodes and objects; that
way, programming languages like JavaScript can interact with the page.</p>
    </div>

    <!-- Demonstration Area -->
    <div class="demo-area">
      <h2>Demonstration</h2>

      <!-- This is the element we will manipulate using JavaScript -->
      <div id="targetElement" class="box" title="I am a box">
        I am the original text.
      </div>

      <div class="buttons">
        <button id="changeContentBtn">Change Content</button>
        <button id="changeStyleBtn">Change Style</button>
        <button id="changeAttrBtn">Change Attribute</button>
        <button id="resetBtn" class="reset-btn">Reset</button>
      </div>

      <p id="attrStatus" class="status-msg"></p>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>

```

Question4/style.css

```

body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
  background-color: #f9f9f9;
}

```

```
.container {
  max-width: 800px;
  margin: 0 auto;
  background: white;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}

.explanation {
  background-color: #e9ecef;
  padding: 15px;
  border-left: 5px solid #007bff;
  margin-bottom: 30px;
  line-height: 1.6;
}

.demo-area {
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 8px;
}

.box {
  width: 300px;
  height: 100px;
  background-color: lightgray;
  color: black;
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 20px;
  border: 2px solid #999;
  font-weight: bold;
  text-align: center;
  padding: 10px;
  /* transition makes the style changes smooth */
  transition: all 0.4s ease;
}

.buttons {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

button {
  padding: 10px 15px;
  cursor: pointer;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
  font-weight: bold;
}
```

```

}

button:hover {
  background-color: #0056b3;
}

.reset-btn {
  background-color: #dc3545;
}

.reset-btn:hover {
  background-color: #c82333;
}

.status-msg {
  color: #28a745;
  font-size: 0.9em;
  margin-top: 15px;
  font-weight: bold;
}

```

Question4/script.js

```

// 1. Select the target HTML elements using their IDs
const targetElement = document.getElementById('targetElement');
const attrStatus = document.getElementById('attrStatus');

// 2. Select all the buttons
const contentBtn = document.getElementById('changeContentBtn');
const styleBtn = document.getElementById('changeStyleBtn');
const attrBtn = document.getElementById('changeAttrBtn');
const resetBtn = document.getElementById('resetBtn');

// --- DEMONSTRATION 1: Changing Content Dynamically ---
contentBtn.addEventListener('click', () => {
  // We use the textContent property to change the text inside the element
  targetElement.textContent = "Hello! The content was changed dynamically!";
});

// --- DEMONSTRATION 2: Changing Style Dynamically ---
styleBtn.addEventListener('click', () => {
  // We can access inline styles through the .style property
  targetElement.style.backgroundColor = "#28a745"; // Green color
  targetElement.style.color = "white";
  targetElement.style.borderRadius = "30px";
  targetElement.style.border = "4px dashed darkgreen";

  // Also updating text so it matches the vibe
  targetElement.textContent = "Styles applied!";
});

// --- DEMONSTRATION 3: Changing Attributes Dynamically ---
attrBtn.addEventListener('click', () => {

```

```

// Changing the 'title' attribute (hover over the box to see it)
targetElement.setAttribute('title', 'This title was updated by JavaScript!');

// Changing the 'id' or adding an attribute
targetElement.setAttribute('data-modified', 'true');

// Retrieve and display the new attribute to prove it changed
const newTitle = targetElement.getAttribute('title');
attrStatus.textContent = `Attribute updated! Hover over the box. New title attribute
is: "${newTitle}"`;
});

// --- RESET FUNCTIONALITY ---
resetBtn.addEventListener('click', () => {
  // Reset Content
  targetElement.textContent = "I am the original text.";

  // Reset Styles (empty string removes the inline style)
  targetElement.style.backgroundColor = "";
  targetElement.style.color = "";
  targetElement.style.borderRadius = "";
  targetElement.style.border = "";

  // Reset Attributes
  targetElement.setAttribute('title', 'I am a box');
  targetElement.removeAttribute('data-modified');
  attrStatus.textContent = "";
});

```

7. CODE EXPLANATION

Code / Component	Explanation
input.addEventListener('input', fn)	Fires calculation callback immediately whenever the input value changes.
parseFloat(), parseInt()	Parses user string input into decimal floating-point or integer numeric values.
total.toFixed(2)	Formats a numeric amount to standard currency formatting with two decimal places.
readline.createInterface()	Creates terminal I/O streaming interface for interactive user question prompts.
fs.writeFileSync(file, data)	Creates or overwrites a file synchronously on disk with the specified content.

<code>fs.appendFileSync(file, data)</code>	Appends new content synchronously to an existing file without truncating it.
<code>fs.readFileSync(file, 'utf8')</code>	Reads file contents synchronously from disk decoded as UTF-8 plaintext.
<code>express()</code>	Factory function initializing an instance of an Express web application.
<code>app.get(path, handler)</code>	Routes HTTP GET requests targeting a specific URL path to the callback handler.
<code>res.json(data)</code>	Sends a JSON response with proper HTTP headers (application/json) and serialization.
<code>app.listen(port, callback)</code>	Binds and listens for incoming network connections on the designated port.
<code>element.textContent</code>	Gets or sets the text content of the node without HTML parsing overhead.
<code>element.style.property</code>	Directly manipulates inline CSS styles of an element dynamically.
<code>element.setAttribute(attr, val)</code>	Sets the value of a specified attribute on the targeted HTML element.

8. TEST CASES AND EXECUTION DATA

Test Case	Input Name	Test Input	Expected Result	Actual Result	Status
TC-01	Product Calculator	Price: 1200, Qty: 2	Total updates dynamically to \$2400.00 without refresh	Calculated correctly	Pass
TC-02	Default Product Fallback	Empty product name	Displays fallback name 'Product' in total cost display	Fallback displayed	Pass
TC-03	Node File Creation	student_notes.txt	File created and initial content written to disk	File created	Pass
TC-04	Node File Append & Read	Additional Content	Appends content and logs combined file text to console	Logged correctly	Pass

TC-05	Express Route	Student	GET /students	Returns HTTP 200 with JSON array containing 5 student objects	JSON delivered	Pass
TC-06	DOM Modification	Content	Click 'Change Content'	Box text updates to dynamic greeting	Content updated	Pass
TC-07	DOM Modification	Style	Click 'Change Style'	Box background changes to green with 30px border radius	Styles applied	Pass
TC-08	DOM Update	Attribute	Click 'Change Attribute'	Title attribute updated; confirmed via status message	Attribute updated	Pass

9. OUTPUT

Output 1: Interactive Product Calculator Execution

Product Price Calculator

Product Name:

Price (\$):

Quantity:

Total Cost

Total for Gaming Laptop: \$2400.00

Output 2: Node.js File Management Terminal Execution

```
PS C:\Observation Tasks\Task 5\Question2> node app.js
=====
Node.js File Management
=====

Enter a filename (e.g., myData.txt): student_notes.txt
Enter content to write to the file: Full Stack Web Development - Node.js File System Module.
[SUCCESS] File 'student_notes.txt' created/written successfully.

Enter additional content to append to the file: Appended Content: Demonstration of appendFileSync and
[SUCCESS] Additional content appended to 'student_notes.txt'.

--- Final Contents of student_notes.txt ---
Full Stack Web Development - Node.js File System Module.
Appended Content: Demonstration of appendFileSync and readFileSync.
-----
```

Output 3: Express.js Student API Server Endpoint Output

JSON Response from Express API Route:

```
[
  { "id": 1, "name": "Alice Johnson", "course": "Computer Science" },
  { "id": 2, "name": "Bob Smith", "course": "Information Technology" },
  { "id": 3, "name": "Charlie Brown", "course": "Mechanical Engineering" },
  { "id": 4, "name": "Diana Prince", "course": "Electrical Engineering" },
  { "id": 5, "name": "Evan Wright", "course": "Civil Engineering" }
]
```

```
PS C:\Observation Tasks\Task 5\Question3> node app.js
🟢 Server is successfully running!
🟢 Open your browser and visit: http://localhost:3000
```

Output 4: Document Object Model (DOM) Dynamic Modification

Understanding the DOM

What is the DOM?

The Document Object Model (DOM) is a programming interface for web documents...

Styles applied!

Change Content

Change Style

Change Attribute

Reset

Attribute updated! Hover over the box. New title attribute is: "This title was updated by JavaScript"

10. RESULT

Thus, all four Project-Based Learning technical questions for Task 5 were successfully developed, executed, and verified across all test cases. Dynamic price calculation, Node.js file system management, Express REST routing, and comprehensive DOM manipulation functioned without errors.